TLS Implementation with RSA, ECDHE & AES

Akshay Malwey
Rohit Khatri

Information Security – Assignment 1
Supervisor: Dr. Gaurav Varshney

https://github.com/Akshaymalwey/TLS-1.3-from-scratch

Project Workflow Summary

1. **Key & Certificate Setup**
   - Generated RSA public/private key pairs using OpenSSL
   - Created three entities: Client, Server, and self-signed Root CA
   - Root CA issued and signed certificates for both Client and Server
2. **TLS-like Handshake**
   - Client sends **ClientHello** containing:
     - Supported curve (prime256v1)
     - Client random value
     - Client ephemeral public key (ECDHE)
   - Server responds with **ServerHello** containing:
     - Server random value
     - Server certificate (signed by Root CA)
     - Server ephemeral public key
     - RSA signature over handshake parameters
   - Client verifies certificate chain and signature authenticity
3. **Key Exchange & Secure Communication**
   - Both sides compute shared secret using ECDHE
   - Session keys derived using HKDF with SHA-256
   - Secure communication established using AES-256-GCM

Code Implementation Snippets

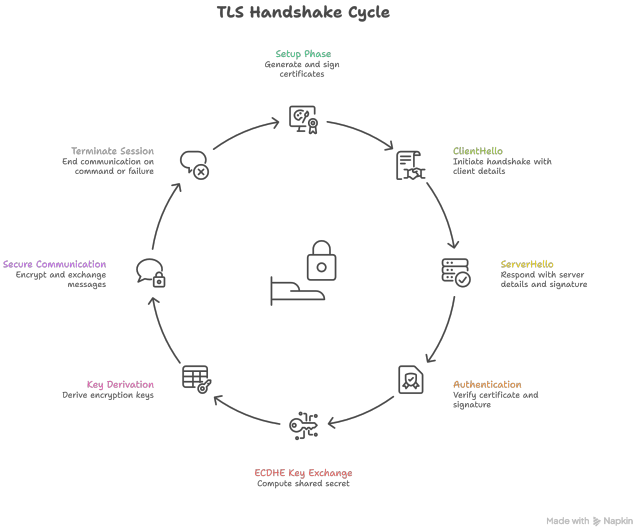
```c
//hkdfc key generation
const sharedSecret = ecdh.computeSecret(clientEphemeralPk)
function deriveSessionKeys(sharedSecret, clientRandom, serverRandom){
    const salt = Buffer.concat([clientRandom, serverRandom])
    const keyMat = crypto.hkdfSync(
            'sha256',
            sharedSecret,
            salt,
            Buffer.from('handshake data'),
            64
        )

    return {
        clientWKey: keyMat.slice(0,32),
        serverWKey: keyMat.slice(32,64)
    }
}
```

```c
//function to encrypting the message
function encryptMessage(plaintext, key){
    const iv = crypto.randomBytes(12)
    const cipher = crypto.createCipheriv('aes-256-gcm', key, iv)
    const encrypted = Buffer.concat([
        cipher.update(plaintext, 'utf8'),
        cipher.final()
    ])
    const authTag = cipher.getAuthTag();

    return {
        iv: iv.toString('base64'),
        ciphertext: encrypted.toString('base64'),
        tag: authTag.toString('base64')
    }

}
```

```c
//sign the ecdh public key by pvt RSA key of server
const sign = crypto.createSign('sha256')
sign.update(Buffer.concat([clientRandom, serverRandom, serverEphemeralPk]))
sign.end()
const signature = sign.sign(serverKey)
```


```
-----BEGIN CERTIFICATE-----
MIIDazCCAlOgAwIBAgIUJmy43jevrFnGYgTNhw1P2r75a6YwDQYJKoZIhvcNAQEL
BQAwRTELMAkGA1UEBhMCaW4xCzAJBgNVBAgMAmptMQswCQYDVQQKDAJpajELMAkG
A1UECwwCaXMxDzANBgNVBAMMBmFrc2hheTAeFw0yNjAyMjYwNjQxNTlaFw0yNzAy
MjYwNjQxNTlaMEUxCzAJBgNVBAYTAmluMQswCQYDVQQIDAJqbTELMAkGA1UECgwC
aWoxCzAJBgNVBAsMAmlzMQ8wDQYDVQQDDAZha3NoYXkwggEiMA0GCSqGSIb3DQEB
AQUAA4IBDwAwggEKAoIBAQCWd/rJqigM7hautk8r7eBZEiW6WM2W4nIaU05Pi7FP
xBIQehT2/LIz21ifCliigii0rz5RTfKRr5DxOv/SHJ2aOX8e+iCoGeq1hDzV/1L/
8P2bhVLYdV3SR2Vg9PDZx4pqtLTfbnnY77hZYV4inLdt2Fvzn/wBpUq2Wc/2cMym
p8jctGdwQ9AowIUhCWX/BUAeAAxDIFftlhIVU5WSz/J4JPmNFR5rNY+JAx625WAK
+NjjFAx9LwV5PHJVcq9H4V881A/guw+K74sah/7yNO/wO2224YoIcBwgd2x5KUer
yEFiR3cTxQdBfXOF9OVymwSOKzA18E96a6/NBWWePg3rAgMBAAGjUzBRMB0GA1Ud
DgQWBBSx7Eldh1BWw0175XC3gDutwK2sOjAfBgNVHSMEGDAWgBSx7Eldh1BWw017
5XC3gDutwK2sOjAPBgNVHRMBAf8EBTADAQH/MA0GCSqGSIb3DQEBCwUAA4IBAQA4
CnWUfMxHnrDYRFb+jRSH7fzrwVOEszgWp6MVcvYs/zu2kDq+4INHvMSodpObip1O
zaGMNpD9LnM40AS/rBUw9w/Q51w+msqadYt6NQMksAhuJ/GeaCxe12JItSfxw00m
U5t75Nk8c5zMm6LRpZEZpcJq6Dtnr4856QJf8eW6R5u+iVQOnY1uQEQjKTLTM4HD
TaBHZmzWS43tCaO1edR19twHR1oIAeP8TcWOdBoqQPSi9QOj/ggs7+/0k59QgxV
OoOHf0F87C1dAHMKGzisCH49VwxDfwsC0gkUfsHo214p3IIiKwZuOD6quWJvjTj8
qUagz6haLxwW4yCcZHIy
-----END CERTIFICATE-----
```

Complete Workflow



Thank You